TOOLS & SOFTWARE

Eclipse IDE

Master the extensible IDE behind Java and SAP ADT — workspaces, perspectives, debugging, refactoring, and the shortcuts that make it fast.

eclipse · ide · java · sap-adt · debugging · refactoring · productivity

Study Pack · Comprehensive reference & interview prep

CONTENTS

1. What Eclipse Is and Why It Matters
2. Getting Started: Install & First Launch
3. Workspaces & Projects
4. Perspectives, Editors & Views
5. The Java Editor: Features That Matter
6. Build, Run & Launch Configurations
7. Debugging & Breakpoints
8. Refactoring
9. Plugins & the Marketplace
10. Java & SAP ADT Usage
11. Cheat-Sheet: Essential Shortcuts
12. Best Practices & Common Pitfalls
13. Interview Questions & Answers
14. Further Learning

What Eclipse Is and Why It Matters

Eclipse is a free, open-source, extensible Integrated Development Environment (IDE) built on a modular plug-in architecture. It is the foundation beneath Java tooling, SAP's ABAP Development Tools (ADT), and dozens of other language and vendor toolchains — understanding it well pays off across enterprise and interview contexts alike.

At its heart Eclipse is not "a Java editor" — it is a **platform**. The small Equinox runtime (an implementation of the OSGi module system) boots a bus of plug-ins, and everything you see — editors, views, the compiler, the debugger — is a plug-in contributed on top. This is why the same application can host Java (JDT), C/C++ (CDT), web tooling, and SAP ADT simultaneously.

OPEN & FREE

Eclipse Public License (EPL). No per-seat cost, strong enterprise adoption, long-term releases every quarter.

EXTENSIBLE

OSGi/Equinox plug-in model. The Marketplace adds language support, linters, Git clients, and vendor tools.

VENDOR PLATFORM

SAP ADT, Spring Tool Suite, and many commercial IDEs ship *as* Eclipse distributions.

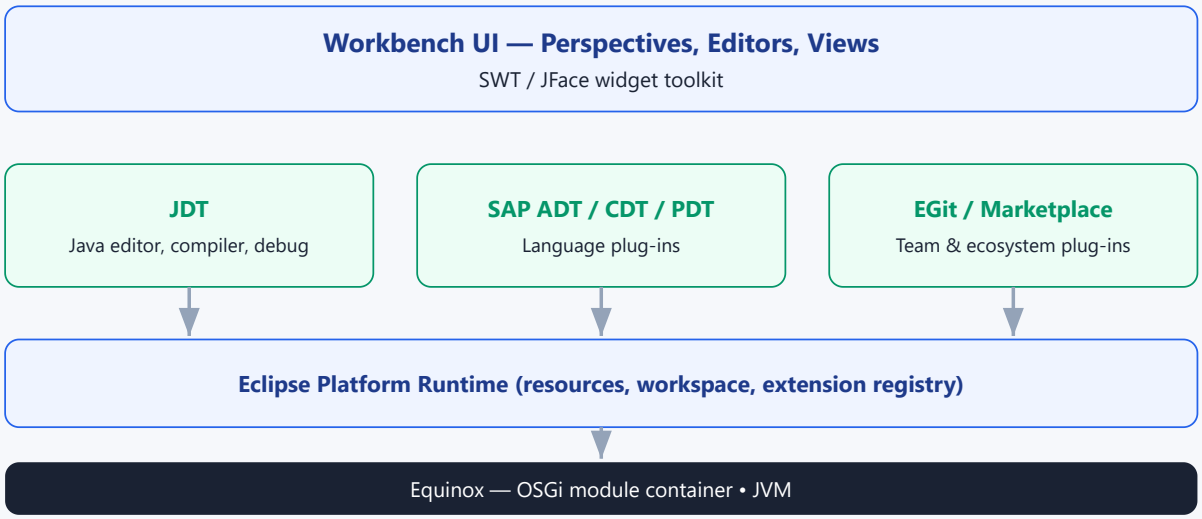


Fig 1. The layered Eclipse architecture: a JVM runs the Equinox OSGi container, which boots the Platform Runtime, which hosts language plug-ins (JDT, ADT, CDT) and the Workbench UI.

Mental model

Think "operating system for developer tools." The Platform provides the kernel (resources, extension points); plug-ins are the installable apps. SAP ADT is just an especially large, well-integrated set of apps.

Getting Started: Install & First Launch

The easiest entry point is the **Eclipse Installer** (Oometo/"Eclipse Installer") which lets you pick a packaged distribution. Common packages:

Package	Contains	Use for
Eclipse IDE for Java Developers	JDT, Git, Maven, Gradle (Buildship)	General Java/backend work
Eclipse IDE for Enterprise Java and Web	Above + WTP, servers, JPA, REST	Jakarta EE / web apps
Eclipse IDE for C/C++ (CDT)	CDT, GDB integration	Native development
ABAP Development Tools (ADT)	SAP plug-ins on an Eclipse base	ABAP / SAP BTP ABAP environment

BASH

```bash
# macOS/Linux: unpack and launch
tar -xzf eclipse-java-2026-03-R-linux-gtk-x86_64.tar.gz
cd eclipse && ./eclipse

# Point Eclipse at a specific JDK by editing eclipse.ini (BEFORE the -vmargs line):
#   -vm
#   /opt/jdk-21/bin/java

# Launch into a specific workspace without the chooser dialog
./eclipse -data /home/me/workspaces/payments

# Clean the OSGi cache if plug-ins misbehave after an install
./eclipse -clean
```

✦ **eclipse.ini**

Most startup problems (wrong JVM, out-of-memory, slow GC) are fixed in `eclipse.ini`. Keep `-vm` explicit so Eclipse never guesses the JDK. Bump heap with `-Xmx2g` after `-vmargs`.

Workspaces & Projects

A **workspace** is a directory that holds your projects plus a hidden `.metadata/` folder containing all your settings, indexes, and view layouts. A **project** is a folder inside (or linked into) the workspace with Eclipse metadata files. This two-level model is the single most important concept to internalize.

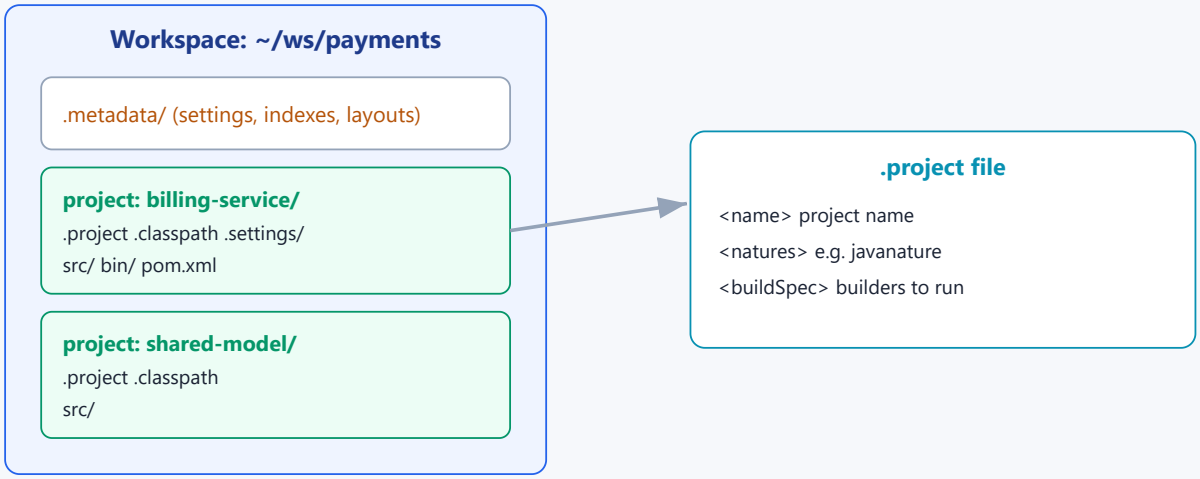


Fig 2. A workspace contains projects; each project carries .project (natures + builders) and .classpath. The workspace .metadata holds UI state and preferences.

Key metadata files

File	Role	Commit to VCS?
`.project`	Project name, natures, builders	Often yes
`.classpath`	Source folders, output, library/dependency entries	Debatable (Maven/Gradle regenerate it)
`.settings/*.prefs`	Per-project compiler/formatter settings	Yes — keeps teams consistent
`.metadata/`	Workspace-wide state (never shared)	Never

> ⚠ **Maven/Gradle caveat**
> With m2e or Buildship, the `.classpath` is *generated* from `pom.xml` / `build.gradle`. Edit the build file, not the classpath, and use `Alt`+`F5` (Maven → Update Project) to resync.

A **nature** tags a project with capabilities (Java, Maven, ABAP). A **builder** is an incremental process triggered on save/change — the Java builder compiles only the changed compilation units and their dependents, which is why Eclipse feels instant compared to a full command-line rebuild.

WORKING SETS

Large workspaces (50+ projects) become unmanageable. **Working sets** are named groups you filter the Package Explorer by — e.g. "payments-core", "payments-tests" — without physically moving

projects.

Perspectives, Editors & Views

Three concepts control the entire UI:

VIEW

A panel showing one thing: Package Explorer, Console, Problems, Outline, Debug, Call Hierarchy. Non-modal, dockable.

EDITOR

Where you edit a resource. Multiple can tile. The center of the workbench; dirty state shown with a `*`.

PERSPECTIVE

A saved arrangement of views + editors tuned for a task: *Java*, *Debug*, *Git*, *ABAP*.

Open resource -> **Editor opens** -> **Switch perspective** -> **Views rearrange**

Switch perspectives from the top-right toolbar or `Window` → `Perspective`. The **Debug** perspective auto-activates when you hit a breakpoint; the **Java** perspective is where you spend most time. Reset a mangled layout with `Window` → `Perspective` → `Reset Perspective`.

✦ **Quick Access**

Press `Ctrl`+`3` (`Cmd`+`3`) to fuzzy-search *any* command, view, preference, or file. It is the single most powerful navigation key — learn it before any other shortcut.

The Java Editor: Features That Matter

The JDT editor is a semantic editor — it understands your code's AST, not just text. That unlocks content assist, quick fixes, and refactoring.

Content assist & code generation

JAVA

```java
public class Order {
    private String id;
    private List<LineItem> items;
    private BigDecimal total;

    // Alt+Shift+S opens the Source menu:
    //  - Generate Getters and Setters
    //  - Generate Constructor using Fields
    //  - Generate hashCode() and equals()
    //  - Generate toString()
    //  - Override/Implement Methods ...

    // Ctrl+Space = content assist. Type the start of a method and complete.
    // "sysout" + Ctrl+Space  →  System.out.println();   (a template)
    // "foreach" + Ctrl+Space →  expands to an enhanced for loop
}
```

Quick Fix & Quick Assist

A lightbulb/red marker means Quick Fix is available. Place the caret on the error and press `Ctrl` + `1`:

Situation	Ctrl+1 offers
Unresolved type	Add import / create class / create interface
Unhandled exception	Surround with try/catch, or add throws clause
Unused variable	Remove, or assign to new local
Method does not exist	Create method in target type with inferred signature
Assignment expression	Assign to new local variable / field (Quick Assist)

Navigation

Action	Shortcut
Open Type (any class on classpath)	`Ctrl` + `Shift` + `T`
Open Resource (any file)	`Ctrl` + `Shift` + `R`
Jump to declaration	`F3` (or Ctrl+click)
Open Type Hierarchy	`F4`
Open Call Hierarchy	`Ctrl` + `Alt` + `H`
Find References in Workspace	`Ctrl` + `Shift` + `G`
Quick Outline (in-file jump)	`Ctrl` + `O`
Last edit location	`Ctrl` + `Q`
Back / Forward navigation	`Alt` + `Left` / `Alt` + `Right`

Build, Run & Launch Configurations

Eclipse builds continuously (**Project** → **Build Automatically** is on by default). Running creates a **launch configuration** — a reusable, named recipe stored under Run → Run Configurations.

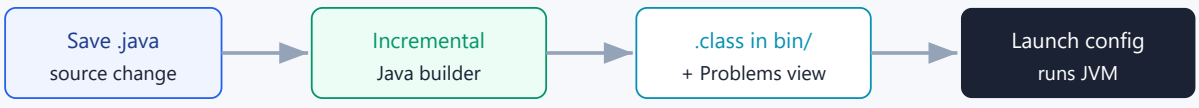


Fig 3. Save-driven incremental build: only changed units recompile, errors land in the Problems view, and a launch configuration starts the JVM.

Action	Shortcut
Run last launched	`Ctrl` + `F11`
Debug last launched	`F11`
Run current class/test (context)	`Alt` + `Shift` + `X`, then `J` (Java) / `T` (JUnit)

A launch config captures: main class, program arguments, VM arguments (`-Xmx` , system properties), environment variables, classpath, and the JRE. Export it to a file so teammates share the same run setup.

Debugging & Breakpoints

Eclipse's debugger is a standout feature. Set a breakpoint by double-clicking the editor gutter, then launch in Debug mode. Execution halts and the Debug perspective opens with Variables, Breakpoints, and the call stack.

Breakpoint types

LINE BREAKPOINT

Halts at a line. Add a **condition** (e.g. `order.getTotal().signum() < 0`) so it only triggers when it matters, or a **hit count**.

EXCEPTION BREAKPOINT

Breaks when a given exception is thrown — caught, uncaught, or both. Priceless for "where does this NPE originate?"

METHOD BREAKPOINT

Breaks on entry/exit of a method, even in libraries without source.

WATCHPOINT (FIELD)

Breaks when a field is read or modified — find who mutates shared state.

Debug action	Shortcut	Meaning
Step Into	`F5`	Enter the called method
Step Over	`F6`	Execute line, don't descend
Step Return	`F7`	Run to end of current method
Resume	`F8`	Continue to next breakpoint
Toggle breakpoint	`Ctrl` + `Shift` + `B`	On the current line

JAVA

```java
public BigDecimal applyDiscount(Order order, Discount d) {
    BigDecimal base = order.getTotal();           // breakpoint here
    BigDecimal factor = d.rate();                 // Variables view shows 'base'
    // Select the expression below and press Ctrl+Shift+I to "Inspect"
    // or Ctrl+Shift+D to "Display" its live value without stopping to log.
    return base.multiply(BigDecimal.ONE.subtract(factor));
}
```

✦ **Power moves**

Drop to Frame re-runs the current method from the top without restarting. **Hot Code Replace** lets you edit a method body while stopped and continue with the new code (works for method-body changes, not signature/field changes). Use **Expressions** view to evaluate arbitrary Java against the paused state.

⚠ **Conditional breakpoint cost**

A condition is evaluated on *every* pass of that line — expensive conditions in hot loops slow the target JVM noticeably. Prefer hit-count or a narrower location when possible.

Refactoring

Because the editor is AST-aware, refactorings are safe and workspace-wide — they update every reference, not just text matches. The refactor menu is `Alt` + `Shift` + `T`.

Refactoring	Shortcut	What it does
Rename	`Alt` + `Shift` + `R`	Rename symbol + all references, incl. getters/JavaDoc
Extract Method	`Alt` + `Shift` + `M`	Pull selection into a new method, infer params/returns
Extract Local Variable	`Alt` + `Shift` + `L`	Introduce a variable for the selected expression
Inline	`Alt` + `Shift` + `I`	Replace a variable/method with its body
Change Method Signature	`Alt` + `Shift` + `C`	Add/reorder/remove params; updates all callers
Move	`Alt` + `Shift` + `V`	Move class/member to another package/type

Show: Extract Method worked example

✓ **Preview first**
Every refactoring offers a Preview with a diff of affected files. On a large codebase, always preview Rename/Change Signature before applying — it catches surprising reflective or test-only references.

Plugins & the Marketplace

Extend Eclipse via **Help** → **Eclipse Marketplace** (drag-and-drop install) or **Help** → **Install New Software** (point at a p2 update site URL). Everything is OSGi bundles resolved by the p2 provisioning system.

Plug-in	Adds
EGit	Git integration (usually pre-bundled)
Buildship	Gradle import, tasks, sync
m2e	Maven import and lifecycle integration
SonarLint	On-the-fly static analysis / code smells
Spring Tools 4	Spring Boot support, live beans, YAML assist
ABAP Development Tools	SAP ABAP + BTP ABAP environment

> **Install vs. restart**
> p2 computes a dependency-consistent set and typically requires a restart. If an install fails, uncheck "Contact all update sites" or verify the target release train (e.g. `2026-03`) matches your Eclipse version.

Java & SAP ADT Usage

ABAP Development Tools (ADT) is SAP's modern, Eclipse-based front end for ABAP, replacing much of the classic SAP GUI SE80 workflow. It installs as plug-ins onto a standard Eclipse Java package via the SAP update site.

ADT concepts mapped to Eclipse

Eclipse concept	ADT meaning
Project	An **ABAP Project** bound to a system connection + client + user
Perspective	The **ABAP** perspective (Project Explorer, Outline, Problems, ABAP Console)
Editor	Source-based editors for classes, programs, CDS views, Core Data Services, Behavior Definitions
Run/Debug	`F8` runs an ABAP program/console; the ABAP debugger attaches on the server
Marketplace/update site	`tools.hana.ondemand.com/[release]` p2 site for ADT

BASH

```bash
# Typical ADT install:
# Help → Install New Software → Add...
#   Name:     ADT
#   Location: https://tools.hana.ondemand.com/latest
# Select "ABAP Development Tools" → Next → accept → restart
# Then: File → New → ABAP Project → choose system from SAP Logon / service key
```

ABAP

```abap
CLASS zcl_greeter DEFINITION PUBLIC CREATE PUBLIC.
  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun.   " enables F8 "ABAP Application (Console)"
ENDCLASS.

CLASS zcl_greeter IMPLEMENTATION.
  METHOD if_oo_adt_classrun~main.
    out->write( |Hello from ADT at { sy-datum }| ).
  ENDMETHOD.
ENDCLASS.
```

In ADT you still get Eclipse's refactoring UX (rename, extract), content assist (`Ctrl`+`Space`), quick fix (`Ctrl`+`1`), and the same navigation keys. ABAP activation (the server-side "compile") is `Ctrl`+`F3`; "Where-Used List" is `Ctrl`+`Shift`+`G`.

✦ **For SAP engineers**

The muscle memory transfers both ways: shortcuts learned for Java navigation and refactoring work almost identically in ADT, so investing in Eclipse fluency pays off across your whole SAP toolchain.

Cheat-Sheet: Essential Shortcuts

Category	Action	Windows/Linux
Navigate	Quick Access (do anything)	`Ctrl` + `3`
Navigate	Open Type / Resource	`Ctrl` + `Shift` + `T` / `R`
Edit	Content assist	`Ctrl` + `Space`
Edit	Quick Fix	`Ctrl` + `1`
Edit	Format source	`Ctrl` + `Shift` + `F`
Edit	Organize imports	`Ctrl` + `Shift` + `O`
Edit	Comment / uncomment line	`Ctrl` + `/`
Edit	Move line up/down	`Alt` + `Up` / `Down`
Edit	Duplicate line	`Ctrl` + `Alt` + `Down`
Edit	Delete line	`Ctrl` + `D`
Source	Source menu / Generate	`Alt` + `Shift` + `S`
Refactor	Refactor menu / Rename	`Alt` + `Shift` + `T` / `R`
Run	Run / Debug last	`Ctrl` + `F11` / `F11`
Search	Find References	`Ctrl` + `Shift` + `G`
Search	File/text search dialog	`Ctrl` + `H`

macOS

Swap `Ctrl` → `Cmd` for most of these; `F3` / `F4` may need Fn depending on keyboard settings.

Best Practices & Common Pitfalls

Do

Share `.settings/` (formatter + compiler level + save actions) via VCS so the whole team's diffs stay clean. Enable Save Actions: format + organize imports on save.

Do

Let the build tool own dependencies (Maven/Gradle). Treat Eclipse's view as a projection of the build model, resynced with Update Project.

Tip

One workspace per unrelated effort keeps indexes small and startup fast. Use Working Sets to tame a big workspace rather than opening everything at once.

Pitfall

Committing `.metadata/` or machine-specific `.classpath` entries. Add them to `.gitignore`; they contain absolute paths and personal UI state.

Pitfall

A corrupted workspace index causes phantom errors ("red" code that compiles fine). Fix with **Project** → **Clean...**, or launch with `-clean`, or (last resort) delete `.metadata/.plugins/org.eclipse.core.resources/.snap`.

Pitfall

Wrong JRE/compiler compliance level produces confusing errors (e.g. records "not supported"). Check **Project** → **Properties** → **Java Build Path** and **Java Compiler**.

Interview Questions & Answers

What is the difference between a workspace and a project in Eclipse?

A workspace is a top-level directory holding one or more projects plus a `.metadata/` folder of IDE-wide settings, indexes, and UI layout. A project is a folder with Eclipse metadata (`.project` , `.classpath` , `.settings/`) describing its natures, builders, and build path. Settings cascade: workspace defaults can be overridden per project.

How does Eclipse achieve such fast builds?

The Java builder is *incremental*. It tracks dependencies between compilation units and recompiles only changed units and their dependents when you save, rather than rebuilding everything. State is cached in the workspace. "Build Automatically" keeps the Problems view live without an explicit compile step.

What is a perspective, and how does it relate to views and editors?

A perspective is a named, saved arrangement of views and editor area tuned for a task (Java, Debug, Git, ABAP). Views are supporting panels (Package Explorer, Console); editors edit resources in the central area. Switching perspectives rearranges views but keeps open editors.

You have an intermittent NullPointerException deep in a loop. How do you debug it efficiently in Eclipse?

Set a Java exception breakpoint on `NullPointerException` (Breakpoints view → Add Java Exception Breakpoint) so execution halts exactly where it's thrown, even uncaught. Alternatively, set a conditional line breakpoint that triggers only when the suspect reference is null. Then inspect the stack, use the Expressions view, and Drop to Frame to replay.

What does the OSGi/Equinox foundation give Eclipse?

A dynamic module system: plug-ins are OSGi bundles with versioned dependencies that can be installed, started, stopped, and resolved at runtime via the p2 provisioning system. This modularity is why the same IDE hosts wildly different toolchains (JDT, CDT, SAP ADT) and why the Marketplace works.

Which files should and shouldn't be committed to version control?

Commit `.settings/` (formatter, compiler compliance, save actions) to standardize the team. `.project` is usually fine. Be cautious with `.classpath` if Maven/Gradle regenerates it. Never commit `.metadata/` — it holds machine-specific UI state and absolute paths.

Rename refactoring vs. find-and-replace — why does it matter?

Rename is AST-aware: it updates every binding reference across the workspace (including qualified names, imports, and optionally JavaDoc and getters/setters) while leaving unrelated same-named

> symbols untouched. Textual find-and-replace is blind to scope and binding, so it both misses and over-matches. Rename also offers a preview diff.

> **What is a launch configuration and why export it?**
>
> It's a named, reusable run recipe capturing the main class/entry point, program and VM arguments, environment, classpath, and JRE. Exporting it (or storing it as a shared launch file in the project) lets teammates run with identical settings, avoiding "works on my machine" launch drift.

> **What is Hot Code Replace and its limits?**
>
> While paused in the debugger, you can edit a method body and continue running with the new bytecode — no restart. It relies on the JVM's redefinition support and only works for method-body changes; adding/removing fields, methods, or changing signatures requires a restart.

> **How is SAP's ADT related to Eclipse?**
>
> ADT (ABAP Development Tools) is a set of Eclipse plug-ins installed from SAP's p2 update site onto a standard Eclipse base. It provides an ABAP perspective, source-based editors (classes, CDS, RAP behavior definitions), server-side activation and debugging, and reuses Eclipse's navigation/refactoring/content-assist. It's SAP's modern replacement for much of SE80.

> **Code looks "red" with errors but compiles fine on the command line. What's happening and how do you fix it?**
>
> Usually a stale or corrupted workspace index/build state, or a mismatched compiler compliance level / JRE. Fixes: Project → Clean, Maven/Gradle → Update Project to resync the classpath, verify Java Compiler level matches the source features used, and if needed restart with `-clean` to rebuild OSGi/caches.

> **What is the single most useful keyboard shortcut and why?**
>
> `Ctrl` + `3` (Quick Access). It fuzzy-searches every command, view, preference, perspective, and recent file, so you can invoke anything without hunting through menus — a universal command palette that also teaches you where features live.

Further Learning

- Work through the built-in **Help** → **Welcome** cheat sheets and the official Eclipse Platform / JDT documentation.
- Memorize ten shortcuts a week from the cheat-sheet table above; verify with `Ctrl` + `Shift` + `L` (list all key bindings).
- Learn **Save Actions** and share a team **formatter profile** via `.settings/`.

- Explore **templates** (Preferences → Java → Editor → Templates) and write your own code snippets.
- For SAP: do SAP's ADT tutorials on the BTP ABAP environment and the RAP (RESTful Application Programming) model.
- Dig into the plug-in development side (PDE) if you want to build your own Eclipse extensions on OSGi.
- Compare with IntelliJ IDEA and VS Code to understand each IDE's trade-offs for your stack.